Laxmi Charitable Trust's
Sheth L.U.J College of Arts & Sir M.V. College of Science and Commerce Department of Information Technology (B.Sc.I.T Semester V)

Applied Business Analytics

Practical 5

Roll No.: T050	Name: Aashu Yadav
Class: TYIT	Batch: 1
Date of Assignment: 21/07/26	Date/Time of Submission: 21/07/26

Aim: Probability and Probability Distributions:

a. Simulate a probability experiment (e.g., rolling dice) using programming and calculate the probabilities of different outcomes.

1. Simulate 1,000 dice rolls and display the first 20 outcomes.

```
import numpy as np
import matplotlib.pyplot as plt
from collections import Counter

# 1. Simulate 1,000 dice rolls
rolls = np.random.randint(1, 7, 1000)

print("First 20 dice outcomes:")
print(rolls[:20])

# Frequency of outcomes
frequency = Counter(rolls)
```

2. Calculate the experimental probability of getting each number from 1to 6.
3. Calculate the theoretical probability of getting each number from 1 to 6.

```
# 2 & 3. Experimental and theoretical probability
print("\nNumber | Frequency | Experimental | Theoretical")

for i in range(1, 7):
    exp_prob = frequency[i] / 1000
    theo_prob = 1 / 6

    print(f"{i:6} | {frequency[i]:9} | "
          f"{exp_prob:.4f}       | {theo_prob:.4f}")
```

```
First 20 dice outcomes:
[6 3 4 2 2 6 5 2 4 1 3 1 2 3 4 6 6 2 1 2]

Number | Frequency | Experimental | Theoretical
     1 |       149 | 0.1490       | 0.1667
     2 |       164 | 0.1640       | 0.1667
     3 |       178 | 0.1780       | 0.1667
     4 |       166 | 0.1660       | 0.1667
     5 |       177 | 0.1770       | 0.1667
     6 |       166 | 0.1660       | 0.1667
```

4. Find the probability of getting 1, 2, 3, 4, 5, 6.

```
# 4. Probability of getting 1, 2, 3, 4, 5, 6
print("\nProbability of each number:")
for i in range(1, 7):
    print(f"P({i}) = {frequency[i] / 1000:.4f}")
```

```
Probability of each number:
P(1) = 0.1490
P(2) = 0.1640
P(3) = 0.1780
P(4) = 0.1660
P(5) = 0.1770
P(6) = 0.1660
```

5. Find the probability of getting an even number.

```
# 5. Even number
even_prob = np.mean(rolls % 2 == 0)
print("\nProbability of Even number:", even_prob)
```

6. Find the probability of getting an odd number.

```
# 6. Odd number
odd_prob = np.mean(rolls % 2 != 0)
print("Probability of Odd number:", odd_prob)
```

7. Find the probability of getting a number greater than 4.

```
# 7. Greater than 4
greater4 = np.mean(rolls > 4)
print("Probability of number > 4:", greater4)
```

8. Find the probability of getting a number less than 4.

```
# 8. Less than 4
less4 = np.mean(rolls < 4)
print("Probability of number < 4:", less4)
```

9. Find the probability of getting a number greater than or equal to 4.

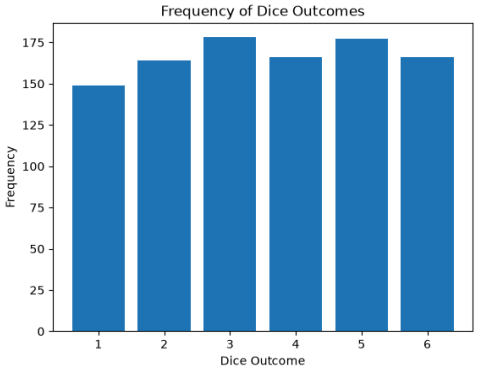
```
# 9. Greater than or equal to 4
greater_equal4 = np.mean(rolls >= 4)
print("Probability of number >= 4:", greater_equal4)
```

Output of 5, 6, 7, 8, 9:

```
Probability of Even number: 0.496
Probability of Odd number: 0.504
Probability of number > 4: 0.343
Probability of number < 4: 0.491
Probability of number >= 4: 0.509
```

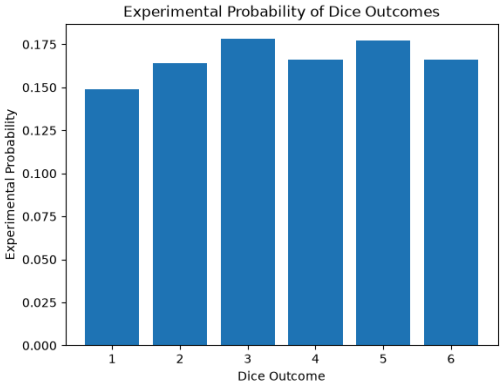
10. Create a bar chart showing the frequency of outcomes 1 to 6.

```
# 10. Bar chart - Frequency
plt.bar(range(1, 7), [frequency[i] for i in range(1, 7)])
plt.xlabel("Dice Outcome")
plt.ylabel("Frequency")
plt.title("Frequency of Dice Outcomes")
plt.show()
```



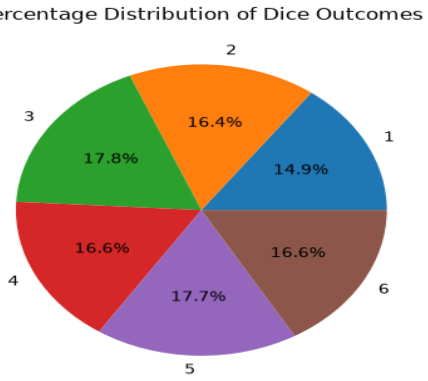
11. Create a bar chart showing the experimental probability of outcomes 1 to 6.

```
# 11. Bar chart - Experimental Probability
experimental = [frequency[i] / 1000 for i in range(1, 7)]

plt.bar(range(1, 7), experimental)
plt.xlabel("Dice Outcome")
plt.ylabel("Experimental Probability")
plt.title("Experimental Probability of Dice Outcomes")
plt.show()
```



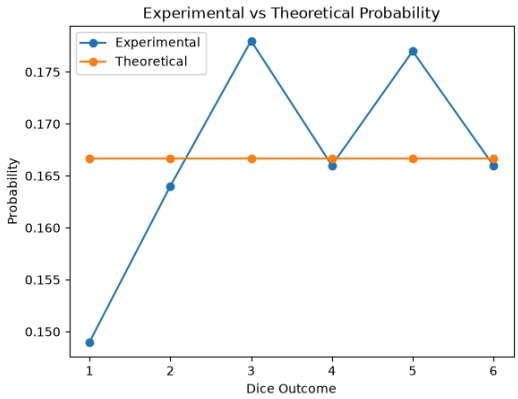
12. Create a pie chart showing the percentage distribution of dice outcomes.

```python
# 12. Pie chart
plt.pie(
    [frequency[i] for i in range(1, 7)],
    labels=range(1, 7),
    autopct="%1.1f%%"
)
plt.title("Percentage Distribution of Dice Outcomes")
plt.show()
```



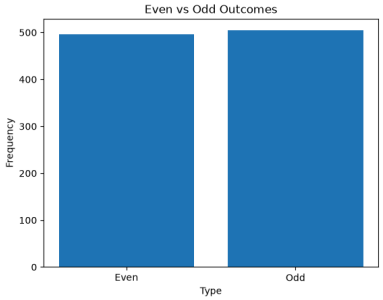
13. Create a line chart comparing experimental probability and theoretical probability for outcomes 1 to 6.

```python
# 13. Experimental vs Theoretical Probability
theoretical = [1 / 6] * 6

plt.plot(range(1, 7), experimental, marker="o", label="Experimental")
plt.plot(range(1, 7), theoretical, marker="o", label="Theoretical")
plt.xlabel("Dice Outcome")
plt.ylabel("Probability")
plt.title("Experimental vs Theoretical Probability")
plt.legend()
plt.show()
```



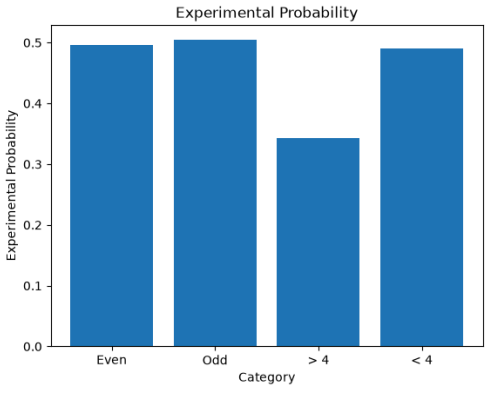
14. Create a chart showing the frequency of Even vs Odd outcomes.

```python
# 14. Even vs Odd frequency
even_count = np.sum(rolls % 2 == 0)
odd_count = np.sum(rolls % 2 != 0)

plt.bar(["Even", "Odd"], [even_count, odd_count])
plt.xlabel("Type")
plt.ylabel("Frequency")
plt.title("Even vs Odd Outcomes")
plt.show()
```



15.Create a chart showing the experimental probability of: • Even numbers • Odd numbers • Numbers greater than 4 • Numbers less than 4

```python
# 15. Experimental probability comparison
categories = ["Even", "Odd", "> 4", "< 4"]
probabilities = [even_prob, odd_prob, greater4, less4]

plt.bar(categories, probabilities)
plt.xlabel("Category")
plt.ylabel("Experimental Probability")
plt.title("Experimental Probability")
plt.show()
```



b. Generate random numbers from various probability distributions (normal, uniform, exponential) and analyze their properties.

1. Generate 1,000 random numbers from a normal distribution with: Mean = 50 and Standard deviation = 10

```
import numpy as np
import matplotlib.pyplot as plt
from collections import Counter

# ---------------- NORMAL DISTRIBUTION ------

normal_data = np.random.normal(50, 10, 1000)
```

2. Calculate the mean, median, mode, variance, and standard deviation of the generated data.

```
# ---------------- NORMAL DISTRIBUTION ------

normal_data = np.random.normal(50, 10, 1000)

mean = np.mean(normal_data)
median = np.median(normal_data)
variance = np.var(normal_data)
std = np.std(normal_data)
minimum = np.min(normal_data)
maximum = np.max(normal_data)
```

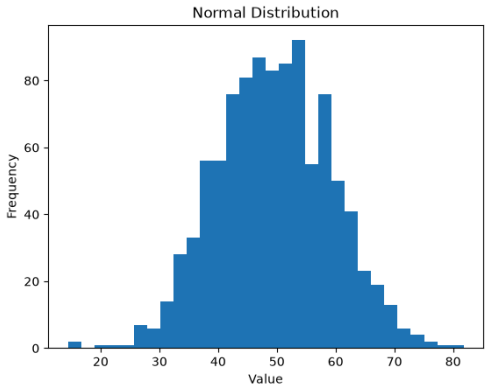
```
NORMAL DISTRIBUTION
Mean: 49.5332960363492
Median: 49.60200648706654
Mode: 40.8
Variance: 95.28837816506365
Standard Deviation: 9.761576622916179
Minimum: 14.427270970536526
Maximum: 81.79236373690101
Percentage between 40 and 60: 69.3 %
Percentage between 30 and 70: 96.1 %
```

3. Calculate the following statistics for the generated Normal distribution: • Mean • Median • Variance • Standard deviation • Minimum • Maximum

```
# Mode
rounded = np.round(normal_data, 1)
mode = Counter(rounded).most_common(1)[0][0]

print("NORMAL DISTRIBUTION")
print("Mean:", mean)
print("Median:", median)
print("Mode:", mode)
print("Variance:", variance)
print("Standard Deviation:", std)
print("Minimum:", minimum)
print("Maximum:", maximum)
```

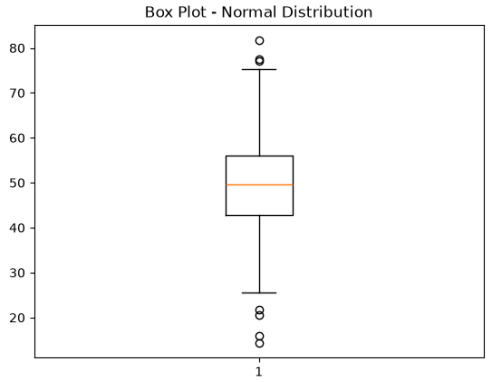
4. Create a histogram of the Normal distribution and analyze its shape.

```
# Histogram
plt.hist(normal_data, bins=30)
plt.xlabel("Value")
plt.ylabel("Frequency")
plt.title("Normal Distribution")
plt.show()
```



5. Create a box plot of the Normal distribution.

```
# Box plot
plt.boxplot(normal_data)
plt.title("Box Plot - Normal Distribution")
plt.show()
```



6. Calculate the percentage of values that fall between: 40<=X<=60

```
# 40 <= X <= 60
p_40_60 = np.mean((normal_data >= 40) & (normal_data <= 60)) * 100
print("Percentage between 40 and 60:", p_40_60, "%")
```

7. Calculate the percentage of values that fall between: 30<=X<=70

```
# 30 <= X <= 70
p_30_70 = np.mean((normal_data >= 30) & (normal_data <= 70)) * 100
print("Percentage between 30 and 70:", p_30_70, "%")
```

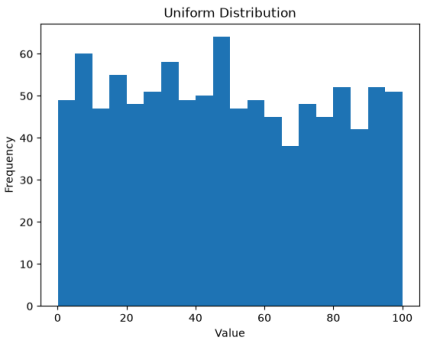
8. Generate 1,000 random numbers from a uniform distribution between 0 and 100.

```
# ---------------- UNIFORM DISTRIBUTION ----------------

uniform_data = np.random.uniform(0, 100, 1000)

print("\nUNIFORM DISTRIBUTION")
print("Mean:", np.mean(uniform_data))
print("Median:", np.median(uniform_data))
print("Standard Deviation:", np.std(uniform_data))
```

```
UNIFORM DISTRIBUTION
Mean: 48.99761018220925
Median: 47.90205296945824
Standard Deviation: 28.912023205486012
```

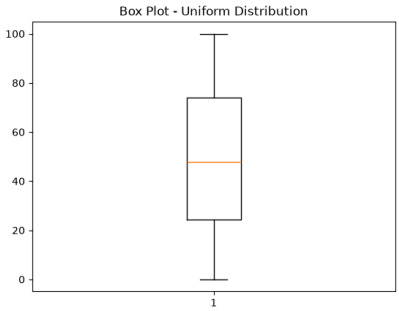
9. Create a histogram of the Uniform distribution.

```
# Histogram
plt.hist(uniform_data, bins=20)
plt.xlabel("Value")
plt.ylabel("Frequency")
plt.title("Uniform Distribution")
plt.show()
```



10. Create a box plot of the Uniform distribution.

```
# Box plot
plt.boxplot(uniform_data)
plt.title("Box Plot - Uniform Distribution")
plt.show()
```



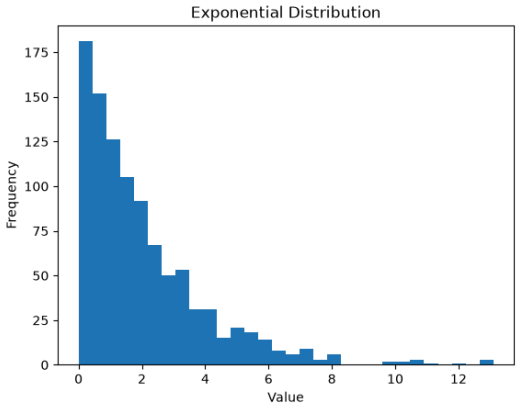
11. Generate 1,000 random numbers from an Exponential distribution with scale = 2.

```
# ---------------- EXPONENTIAL DISTRIBUTION ----------------

exponential_data = np.random.exponential(scale=2, size=1000)

print("\nEXPONENTIAL DISTRIBUTION")
print("Mean:", np.mean(exponential_data))
print("Median:", np.median(exponential_data))
print("Standard Deviation:", np.std(exponential_data))
```

```
EXPONENTIAL DISTRIBUTION
Mean: 2.043933702194961
Median: 1.4439336751788021
Standard Deviation: 1.9846720126915929
```

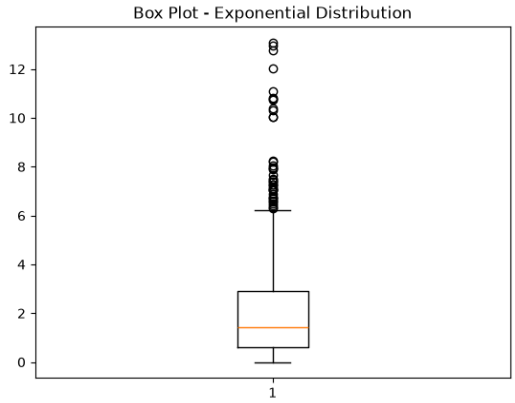
12. Create a histogram of the Exponential distribution.

```
# Histogram
plt.hist(exponential_data, bins=30)
plt.xlabel("Value")
plt.ylabel("Frequency")
plt.title("Exponential Distribution")
plt.show()
```



13. Create a box plot and identify possible extreme values.

```
# Box plot
plt.boxplot(exponential_data)
plt.title("Box Plot - Exponential Distribution")
plt.show()
```



14. Calculate the percentage of values greater than 2.

```
# Percentage greater than 2
greater2 = np.mean(exponential_data > 2) * 100
print("Percentage greater than 2:", greater2, "%")
```

15. Calculate the percentage of values greater than 5.

```
# Percentage greater than 5
greater5 = np.mean(exponential_data > 5) * 100
print("Percentage greater than 5:", greater5, "%")
```

Output of 14, 15:

```
Percentage greater than 2: 38.1 %
Percentage greater than 5: 8.1 %
```

c. Coin toss task

1. Simulate 1,000 coin tosses and calculate the experimental probability of: • Getting Heads • Getting Tails

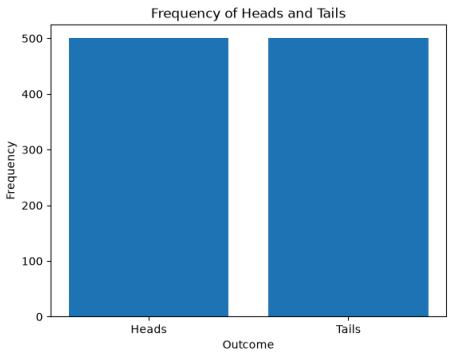
```python
import numpy as np
import matplotlib.pyplot as plt

# ---------------- 1,000 COIN TOSSES ----------------

tosses = np.random.choice(["Heads", "Tails"], 1000)

heads = np.sum(tosses == "Heads")
tails = np.sum(tosses == "Tails")

exp_heads = heads / 1000
exp_tails = tails / 1000

theo_heads = 0.5
theo_tails = 0.5

print("COIN TOSS - 1,000 TOSSES")
print("Heads:", heads)
print("Tails:", tails)

print("Experimental Probability of Heads:", exp_heads)
print("Experimental Probability of Tails:", exp_tails)

print("Theoretical Probability of Heads:", theo_heads)
print("Theoretical Probability of Tails:", theo_tails)
```

2. Calculate the theoretical probability of Heads and Tails.

```
COIN TOSS - 1,000 TOSSES
Heads: 500
Tails: 500
Experimental Probability of Heads: 0.5
Experimental Probability of Tails: 0.5
Theoretical Probability of Heads: 0.5
Theoretical Probability of Tails: 0.5
```

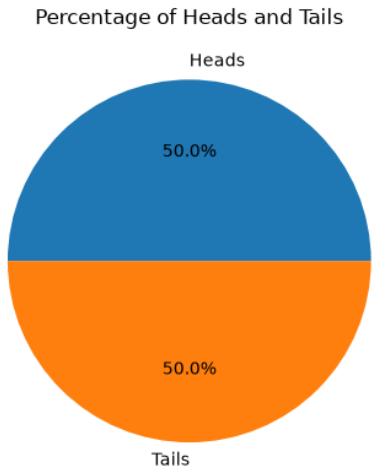
3. Create a bar chart showing the frequency of Heads and Tails.

```python
# 3. Bar chart
plt.bar(["Heads", "Tails"], [heads, tails])
plt.xlabel("Outcome")
plt.ylabel("Frequency")
plt.title("Frequency of Heads and Tails")
plt.show()
```



4. Create a pie chart showing the percentage of Heads and Tails.

```python
# 4. Pie chart
plt.pie([heads, tails], labels=["Heads", "Tails"], autopct="%1.1f%%")
plt.title("Percentage of Heads and Tails")
plt.show()
```



5. If a coin is tossed 10 times and Heads occurs 8 times, calculate the experimental probability of Heads.

```python
# 5. 10 tosses, 8 Heads
experimental_probability = 8 / 10

print("\n10 Tosses:")
print("Heads = 8")
print("Experimental Probability of Heads:", experimental_probability)
```

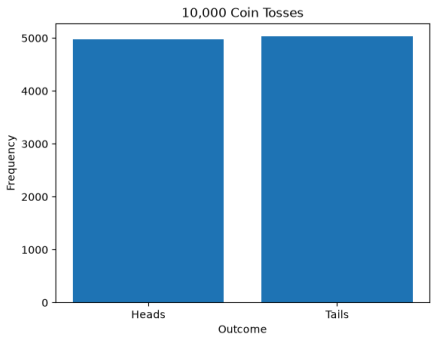
```
10 Tosses:
Heads = 8
Experimental Probability of Heads: 0.8
```

6. Simulate 10,000 coin tosses and create a complete analysis containing: • Total tosses • Number of Heads • Number of Tails • Experimental probability of Heads • Experimental probability of Tails • Theoretical probabilities • Difference between experimental and theoretical probabilities • Bar chart • Pie chart

```python
# --------------- 10,000 COIN TOSSES ---------------

tosses = np.random.choice(["Heads", "Tails"], 10000)

heads = np.sum(tosses == "Heads")
tails = np.sum(tosses == "Tails")

exp_heads = heads / 10000
exp_tails = tails / 10000

print("\nCOIN TOSS - 10,000 TOSSES")
print("Total Tosses:", 10000)
print("Number of Heads:", heads)
print("Number of Tails:", tails)

print("Experimental Probability of Heads:", exp_heads)
print("Experimental Probability of Tails:", exp_tails)

print("Theoretical Probability of Heads:", 0.5)
print("Theoretical Probability of Tails:", 0.5)

print("Difference in Heads Probability:",
    abs(exp_heads - 0.5))

print("Difference in Tails Probability:",
    abs(exp_tails - 0.5))
```

```
COIN TOSS - 10,000 TOSSES
Total Tosses: 10000
Number of Heads: 4973
Number of Tails: 5027
Experimental Probability of Heads: 0.4973
Experimental Probability of Tails: 0.5027
Theoretical Probability of Heads: 0.5
Theoretical Probability of Tails: 0.5
Difference in Heads Probability: 0.00269999999999998
Difference in Tails Probability: 0.0027000000000000357
```

```python
# Bar chart
plt.bar(["Heads", "Tails"], [heads, tails])
plt.xlabel("Outcome")
plt.ylabel("Frequency")
plt.title("10,000 Coin Tosses")
plt.show()
```




```python
# Pie chart
plt.pie([heads, tails],
        labels=["Heads", "Tails"],
        autopct="%1.1f%%")
plt.title("10,000 Coin Tosses - Percentage")
plt.show()
```